SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS: **Total Marks: 25**

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

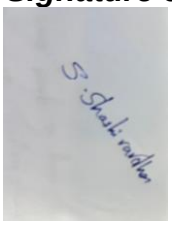
3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

4. An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5. A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I S.Shashi Vardhan Reddy/192311155 Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1. REST vs SOAP and Appropriate Web Service Solution

REST (Representational State Transfer) and SOAP (Simple Object Access Protocol) are two common approaches for developing web services.

Feature	REST	SOAP

Architecture	**Architectural style**	**Protocol**
Data formats	**Mainly JSON, also XML**	**Mainly XML**
Performance	**Faster and lightweight**	**Relatively slower**
Scalability	**Highly scalable**	**More complex to scale**
Security	**HTTPS, OAuth, JWT, etc.**	**WS-Security and HTTPS**
Usage	**Mobile/web applications, public APIs**	**Enterprise and highly formal transactions**

For a mobile and web application, REST is the more appropriate choice. REST APIs are lightweight and usually use JSON, which reduces data size and improves response time. They are also stateless, making it easier to distribute requests across multiple servers and scale horizontally.

Security can be provided using HTTPS/TLS, OAuth 2.0 or OpenID Connect for authentication, and JWTs where appropriate. Rate limiting, input validation, and authorization should also be implemented.

SOAP would be suitable when strict contracts, XML-based messaging, or advanced WS-* standards are specifically required, such as some enterprise or legacy banking integrations.

Conclusion: REST should be selected because it provides better performance, simpler development, easier scalability, and sufficient security for a modern mobile and web application.

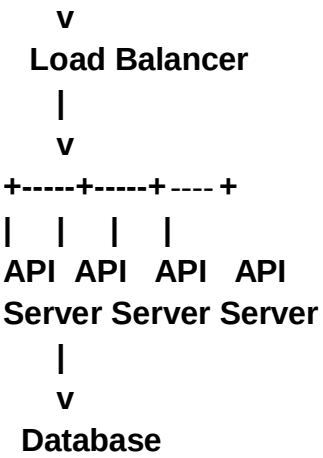
---

2. Scalable Service Architecture

A scalable architecture should distribute requests among multiple application servers rather than relying on a single server.

Basic architecture:

Clients
(Web / Mobile)
|



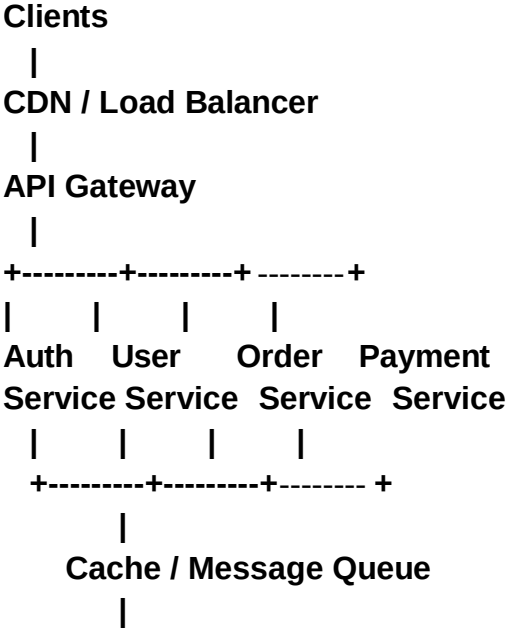
Working

1. **The web or mobile client sends an HTTP/HTTPS request.**
2. **The load balancer receives the request and distributes it among available API servers.**
3. **API servers process business logic and communicate with the database.**
4. **The database stores and retrieves application data.**
5. **The response is returned through the API server to the client.**

This architecture supports horizontal scaling because additional API servers can be added when the number of users increases. Health checks can remove failed servers from the load-balancing pool.

Alternative design

A better design for very large traffic is:



Database Cluster

This microservices-based architecture separates major functions into independent services. Caching reduces repeated database queries, while message queues allow time-consuming operations to be processed asynchronously. Database replication and partitioning can further improve scalability.

For a smaller application, however, a modular monolithic API may be simpler and more cost-effective than microservices.

---

3. Secure Communication Protocol for Banking

Possible communication approaches include HTTP, HTTPS, TCP, and application-level protocols such as SOAP or REST over HTTPS.

HTTPS using TLS (Transport Layer Security) should be selected for communication between the banking client and server.

Banking Client
|
| HTTPS + TLS
v
Banking Server
|
v
Secure Database

Reasons for selecting HTTPS/TLS

- **Security: TLS encrypts data transmitted between client and server, protecting credentials, account information, and transactions from eavesdropping.**
- **Authentication: The server proves its identity using a digital certificate. Mutual TLS can also be used where appropriate.**
- **Integrity: TLS helps prevent attackers from modifying messages during transmission.**

- **Reliability: HTTPS operates over TCP, providing reliable and ordered delivery.**
- **Performance: Modern TLS versions, particularly TLS 1.3, provide strong security with relatively low connection overhead.**

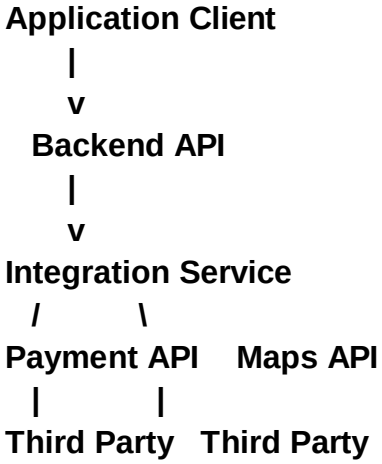
For a banking system, HTTPS should be combined with strong application-level authentication and authorization, such as multi-factor authentication, short-lived access tokens, transaction signing where required, rate limiting, and secure session management.

Conclusion: HTTPS with modern TLS is the most suitable standard protocol because it provides confidentiality, integrity, authentication, reliability, and good performance.

---

4. API Integration Strategy

A web application integrating with payment gateways, map providers, or other third-party services should use a controlled API integration layer.



Request handling

1. **The client sends a request to the application's backend.**
2. **The backend validates the request and required parameters.**
3. **The integration service creates the appropriate third-party API request.**
4. **Authentication credentials are securely attached.**
5. **The third-party service processes the request.**
6. **The response is validated, transformed into the application's format, and returned to the client.**

Authentication

API keys, OAuth 2.0, signed requests, or other mechanisms required by the provider should be used. Credentials must be stored securely in a secrets manager and should never be embedded in client-side code.

Response processing

The integration layer should handle successful responses, validation errors, timeouts, rate limits, and provider failures. Responses can be normalized so that the rest of the application does not depend heavily on a particular third-party API.

Alternative approach

For improved reliability, an asynchronous integration using a message queue can be used for operations that do not require an immediate response.

Caching can also reduce repeated map or data requests, while retries with exponential backoff can handle temporary failures.

---

5. Error Handling and Fault Tolerance

A reliable web service should detect errors, recover where possible, and provide useful responses without exposing internal system information.

Client
|
v
API Gateway
|
v
Service
|
+--- > Retry / Timeout
|
+--- > Circuit Breaker
|
+--- > Logging & Monitoring
|
v
Database / External Service

Error-handling mechanism

1. **Input validation: Reject invalid requests before processing them.**
2. **HTTP status codes: Use appropriate codes such as 400 for bad requests, 401/403 for authentication or authorization failures, 404 for missing resources, and 500/503 for server or temporary service failures.**
3. **Timeouts: Prevent a slow external service from blocking resources indefinitely.**
4. **Retries: Retry temporary failures using exponential backoff, with a limit on retry attempts.**
5. **Circuit breaker: Temporarily stop requests to a repeatedly failing service and allow it time to recover.**
6. **Logging and monitoring: Record errors, latency, and service health for diagnosis.**
7. **Graceful degradation: Provide a limited but useful service when a non-critical dependency fails.**

Alternative strategies

- **Redundancy: Run multiple instances of critical services.**
- **Load balancing: Distribute requests among healthy servers.**
- **Caching: Continue serving frequently requested data when appropriate.**
- **Message queues: Prevent temporary downstream failures from causing immediate request failures.**
- **Database replication and backups: Improve availability and recovery.**
- **Health checks and automatic failover: Detect failed components and redirect traffic.**

Conclusion: Combining validation, timeouts, controlled retries, circuit breakers, monitoring, redundancy, and graceful degradation provides strong fault tolerance and a smoother user experience.